

26. Find a Turing machine to compute the function

$$f(n_1, n_2) = \max(n_1, n_2)$$

27. Do Exercise 21 again, this time making use of the machines T_1 and T_2 of Exercises 16 and 23, respectively, as “functions.” (Formally, the states of these machines would have to be renumbered as the quintuples are inserted into the “main program,” but you may omit this tiresome detail and merely “invoke T_1 ” or “invoke T_2 .”)
28. Describe verbally the actions of a Turing machine that computes the function $f(n_1, n_2) = n_1 \cdot n_2$, that is, design the algorithm but do not bother to create all the necessary quintuples. You may make use of Exercises 16 and 23.

SECTION 9.5 FORMAL LANGUAGES

Suppose we come upon the English language sentence, “The walrus talks loudly.” Although we might be surprised at the meaning, or *semantics*, of the sentence, we accept its form, or *syntax*, as valid in the language, meaning that the various parts of speech (noun, verb, and so on) are strung together in a reasonable way. In contrast, we reject, “Loudly walrus the talks,” as an illegal combination of parts of speech, or as syntactically incorrect and not part of the language. We must also worry about correct syntax in programming languages, but in these, unlike natural languages (English, French, and so on), legal combinations of symbols are specified in detail. Let’s give a formal definition of *language*; the definition will be general enough to include both natural languages and programming languages.

DEFINITIONS ALPHABET, VOCABULARY, WORD, LANGUAGE

An **alphabet** or **vocabulary** V is a finite, nonempty set of symbols. A **word** over V is a finite-length string of symbols from V . The set V^* is the set of all words over V . (See Example 6 in Chapter 3 for a recursive definition of V^* .) A **language** over V is any subset of V^* .

Viewing syntactically correct English language as a subset L of the set of all strings over the usual alphabet, we feel that, “The walrus talks loudly,” belongs to L while, “Loudly walrus the talks,” does not.

For any given language L , how can we describe L , that is, specify exactly those words belonging to L ? If L is finite, we can just list its members, but if L is infinite, can we find a finite description of L ? Not always—there are many more languages than possible finite descriptions. Although we will consider only languages that can be finitely described, we can still think of two possibilities. We may be able to describe an algorithm to *decide* membership in L ; that is, given any word in V^* , we could apply our algorithm and receive a yes or no answer as to whether the word belongs to L . Or we may be able to describe a procedure allowing us only to *generate* members of L , that is, crank out one at a time a list of all the members of L . We will settle for languages for which this second option is possible and describe such a language by defining its generative process, or giving a *grammar* for the language.

Before we give a formal definition of what constitutes a grammar, let's look again at why, "The walrus talks loudly," seems to be an acceptable sentence by seeing how it could be generated. Starting from the notion of sentence, we would agree that one legitimate form for a sentence is a noun phrase followed by a verb phrase. Symbolically,

sentence $\rightarrow$ **noun-phrase verb-phrase**

A legitimate form of noun phrase is an article followed by a noun,

noun-phrase $\rightarrow$ **article noun**

and a legitimate form of verb phrase is a verb followed by an adverb,

verb-phrase $\rightarrow$ **verb adverb**

We would also agree with the substitutions

article $\rightarrow$ the
noun $\rightarrow$ walrus
verb $\rightarrow$ talks
adverb $\rightarrow$ loudly

Thus we can generate the sentence, "The walrus talks loudly," by making successive substitutions:

sentence $\Rightarrow$ **noun-phrase verb-phrase**
 $\Rightarrow$ **article noun verb-phrase**
 $\Rightarrow$ the **noun verb-phrase**
 $\Rightarrow$ the walrus **verb-phrase**
 $\Rightarrow$ the walrus **verb adverb**
 $\Rightarrow$ the walrus talks **adverb**
 $\Rightarrow$ the walrus talks loudly

The foregoing boldface terms are those for which further substitutions can be made. The nonboldface terms stop or terminate the substitution process. These ideas are incorporated in the next definition.

● **DEFINITION PHRASE-STRUCTURE (TYPE 0) GRAMMAR**
 A **phrase-structure grammar (type 0 grammar)** G is a 4-tuple, $G = (V, V_T, S, P)$, where
 V = vocabulary
 V_T = nonempty subset of V called the set of **terminals**
 S = element of $V - V_T$ called the **start symbol**
 P = finite set of **productions** of the form $\alpha \rightarrow \beta$ where α is a word over V containing at least one nonterminal symbol and β is a word over V

EXAMPLE 49

Here is a very simple grammar: $G = (V, V_T, S, P)$ where $V = \{0, 1, S\}$, $V_T = \{0, 1\}$, and $P = \{S \rightarrow 0S, S \rightarrow 1\}$.

The productions of a grammar allow us to transform some words over V into others; the productions can be called rewriting rules.

● **DEFINITION GENERATIONS (DERIVATIONS) IN A LANGUAGE**

Let G be a grammar, $G = (V, V_T, S, P)$, and let w_1 and w_2 be words over V . Then w_1 **directly generates (directly derives)** w_2 , written $w_1 \Rightarrow w_2$, if $\alpha \rightarrow \beta$ is a production of G , w_1 contains an instance of α , and w_2 is obtained from w_1 by replacing that instance of α with β . If $w_1, w_2, \dots, w_n$ are words over V and $w_1 \Rightarrow w_2, w_2 \Rightarrow w_3, \dots, w_{n-1} \Rightarrow w_n$, then w_1 **generates (derives)** w_n , written $w_1 \Rightarrow^* w_n$. (By convention, $w_1 \Rightarrow^* w_1$.)

EXAMPLE 50

In the grammar of Example 49, $00S \Rightarrow 000S$ because the production $S \rightarrow 0S$ has been used to replace the S in $00S$ with $0S$. Also $00S \Rightarrow^* 00000S$.

PRACTICE 62

Show that in the grammar of Example 49, $0S \Rightarrow^* 00001$.

● **DEFINITION LANGUAGE GENERATED BY A GRAMMAR**

Given a grammar G , the **language L generated by G** , sometimes denoted $L(G)$, is the set

$$L = \{w \in V_T^* \mid S \Rightarrow^* w\}$$

In other words, L is the set of all strings of terminals generated from the start symbol.

Notice that once a string w of terminals has been obtained, no productions can be applied to w , and w cannot generate any other words.

The following procedure generates a list of the members of L : Begin with the start symbol S and systematically apply some sequence of productions until a string w_1 of terminals has been obtained; then $w_1 \in L$. Go back to S and repeat this procedure using a different sequence of productions to generate another word $w_2 \in L$, and so forth. Actually, this procedure doesn't quite work because we might start on an infinite sequence of direct derivations that never leads to a string of terminals and thus never contributes a word to our list. Instead, we need to run a number of derivations from S simultaneously (parallel processing), checking on each one after each step and adding the final word to the list of members of L for any that terminate. That way we cannot get stuck waiting indefinitely while unable to do anything else.

REMINDER

Productions may be used in any order. The only requirement is that the left side of the production must appear in the string you are processing.

PRACTICE 63 Describe the language generated by the grammar G of Example 49.

Languages derived from grammars such as we have defined are called **formal languages**. If the grammar is defined first, the language will follow as an outcome of the definition. Alternatively, the language, as a well-defined set of strings, may be given first, and we then seek a grammar that generates it.

EXAMPLE 51

Let L be the set of all nonempty strings consisting of an even number of 1s. Then L is generated by the grammar $G = (V, V_T, S, P)$ where $V = \{1, S\}$, $V_T = \{1\}$, and $P = \{S \rightarrow SS, S \rightarrow 11\}$. A language can be generated by more than one grammar. L is also generated by the grammar $G' = (V', V'_T, S', P')$ where $V' = \{1, S\}$, $V'_T = \{1\}$, and $P' = \{S \rightarrow 1S1, S \rightarrow 11\}$.

PRACTICE 64

- Find a grammar that generates the language $L = \{0^n 10^n \mid n \geq 0\}$.
- Find a grammar that generates the language $L = \{0^n 10^n \mid n \geq 1\}$.

Trying to describe concisely the language generated by a given grammar and defining a grammar to generate a given language can both be quite difficult tasks. We'll look at another example where the grammar is a bit more complicated than any we've seen so far. Don't worry about how you might think up this grammar; just convince yourself that it works.

EXAMPLE 52

Let $L = \{a^n b^n c^n \mid n \geq 1\}$. A grammar generating L is $G = (V, V_T, S, P)$ where $V = \{a, b, c, S, B, C\}$, $V_T = \{a, b, c\}$, and P consists of the following productions:

- | | | | |
|-------------------------|------------------------|------------------------|------------------------|
| 1. $S \rightarrow aSBC$ | 3. $CB \rightarrow BC$ | 5. $bB \rightarrow bb$ | 7. $cC \rightarrow cc$ |
| 2. $S \rightarrow aBC$ | 4. $aB \rightarrow ab$ | 6. $bC \rightarrow bc$ | |

It is fairly easy to see how to generate any particular member of L using these productions. Thus, a derivation of the string $a^2 b^2 c^2$ is

$$\begin{aligned}
 S &\Rightarrow aSBC \\
 &\Rightarrow aaBCBC \\
 &\Rightarrow aaBBCC \\
 &\Rightarrow aabBCC \\
 &\Rightarrow aabbCC \\
 &\Rightarrow aabb cC \\
 &\Rightarrow aabbcc
 \end{aligned}$$

In general, $L \subseteq L(G)$ where the outline of a derivation for any $a^n b^n c^n$ is given below; the numbers refer to the productions used.

$$\begin{aligned}
S &\xRightarrow[1]{*} a^{n-1}S(BC)^{n-1} \\
&\xRightarrow[2]{} a^n(BC)^n \\
&\xRightarrow[3]{*} a^nB^nC^n \\
&\xRightarrow[4]{} a^nbB^{n-1}C^n \\
&\xRightarrow[5]{*} a^nb^nC^n \\
&\xRightarrow[6]{} a^nb^ncC^{n-1} \\
&\xRightarrow[7]{*} a^nb^nc^n
\end{aligned}$$

We must also show that $L(G) \subseteq L$, which involves arguing that some productions must be used before others and that the general derivation shown above is the only sort that will lead to a string of terminals. ●

REMINDER

“Throwing in” productions to get the language L you want often means that you generate more than L . The productions must generate exactly the strings in L .

In trying to invent a grammar to generate the L of Example 52, we might first try to use productions of the form $B \rightarrow b$ and $C \rightarrow c$ instead of productions 4 through 7. Then we would indeed have $L \subseteq L(G)$, but $L(G)$ would also include words such as $a^n(bc)^n$.

Formal languages were developed in the 1950s by linguist Noam Chomsky in an attempt to model natural languages, such as English, with an eye toward automatic translation. However, since a natural language already exists and is quite complex, defining a formal grammar to generate a natural language is very difficult. Attempts to do this for English have been only partially successful.

EXAMPLE 53

We can describe a formal grammar that will generate a very restricted class of English sentences. The terminals in the grammar are the words “the,” “a,” “river,” “walrus,” “talks,” “flows,” “loudly,” and “swiftly,” and the nonterminals are the words **sentence**, **noun-phrase**, **verb-phrase**, **article**, **noun**, **verb**, and **adverb**. The start symbol is **sentence** and the productions are

sentence $\rightarrow$ **noun-phrase** **verb-phrase**
noun-phrase $\rightarrow$ **article** **noun**
verb-phrase $\rightarrow$ **verb** **adverb**
article $\rightarrow$ the
article $\rightarrow$ a
noun $\rightarrow$ river
noun $\rightarrow$ walrus
verb $\rightarrow$ talks
verb $\rightarrow$ flows
adverb $\rightarrow$ loudly
adverb $\rightarrow$ swiftly

We know how to derive, “The walrus talks loudly,” in this grammar. Here is a derivation of, “A river flows swiftly”:

sentence $\Rightarrow$ **noun-phrase verb-phrase**
 $\Rightarrow$ **article noun verb-phrase**
 $\Rightarrow$ **a noun verb-phrase**
 $\Rightarrow$ **a river verb-phrase**
 $\Rightarrow$ **a river verb adverb**
 $\Rightarrow$ **a river flows adverb**
 $\Rightarrow$ **a river flows swiftly**

A few other sentences making various degrees of sense, such as, “a walrus flows loudly,” are also part of the language defined by this grammar. The difficulty of specifying a grammar for English as a whole becomes more apparent when we consider that a phrase such as, “time flies,” can be an instance of either a noun followed by a verb or of a verb followed by a noun. This phrase is “ambiguous” (see Exercises 35 and 36 in this section).

Programming languages are less complex than natural languages, and their syntax often can be described successfully using formal language notation.

EXAMPLE 54

A section of formal grammar to generate identifiers in some programming language could be presented as follows:

identifier $\rightarrow$ **letter**
identifier $\rightarrow$ **identifier letter**
identifier $\rightarrow$ **identifier digit**
letter $\rightarrow$ **a**
letter $\rightarrow$ **b**
 $\vdots$
letter $\rightarrow$ **z**
digit $\rightarrow$ **0**
digit $\rightarrow$ **1**
 $\vdots$
digit $\rightarrow$ **9**

Here the set of terminals is $\{a, b, \dots, z, 0, 1, \dots, 9\}$ and **identifier** is the start symbol.

EXAMPLE 55

A shorthand that can avoid a long listing of productions is called **Backus-Naur form** (BNF). The productions of Example 54 can be given in BNF by three lines (as in Example 7 of Chapter 3):

$\langle \text{identifier} \rangle ::= \langle \text{letter} \rangle \mid \langle \text{identifier} \rangle \langle \text{letter} \rangle \mid \langle \text{identifier} \rangle \langle \text{digit} \rangle$
 $\langle \text{letter} \rangle ::= a \mid b \mid c \mid \dots \mid z$
 $\langle \text{digit} \rangle ::= 0 \mid 1 \mid \dots \mid 9$

In BNF, nonterminals are identified by $\langle \rangle$, the production arrow becomes $::=$, and $|$ stands for “or,” identifying various productions having the same left-hand symbol.

In modern times, BNF notation was originally used to define the programming language Algol (ALGOarithmic Language) in the early 1960s. However, it appears that a similar notation was used between 400 BCE and 200 BCE to describe the rules of Sanskrit grammar.²

EXAMPLE 56

The markup language HTML (Hypertext Markup Language) is a specific language for writing documents that will be transmitted across a network and displayed by a Web browser. The browser parses and interprets the various tags that identify legal elements of an HTML document and displays the elements accordingly. The structure of each HTML element, such as a TABLE element, is fixed; someone writing an HTML document using TABLE elements must follow the structure of a TABLE element as defined in HTML. XML (Extensible Markup Language) is a generalized language for defining specific markup languages, that is, it's a language for defining languages. The elements of an XML document can have whatever structure the user chooses; the user defines these elements in an associated DTD (Document Type Definition). The DTD can be part of the XML document, or it can be in a separate file referenced by the XML document (and as such, available for a number of XML documents to use). The following DTD gives a grammar for an XML document about manufactured parts, and defines the structure of a “PARTSLIST” element, an “ITEM” element, and so forth.

```
<!DOCTYPE PARTSLIST {
  <!ELEMENT PARTSLIST (ITEM+)>
  <!ELEMENT ITEM (PARTNUMBER, USEDINLIST)>
  <!ELEMENT PARTNUMBER (#PCDATA)>
  <!ELEMENT USEDINLIST (USEDIN*)>
  <!ELEMENT USEDIN (MAKE, MODEL)>
  <!ELEMENT MAKE (#PCDATA)>
  <!ELEMENT MODEL (#PCDATA)>
}>
```

The DTD uses notation similar to that for regular expressions, where $*$ means zero or more instances, $+$ means one or more instances, and $?$ means zero instances or one instance. So in the preceding DTD, a PARTSLIST consists of one or more ITEMS, and a USEDINLIST consists of zero or more USEDINs. The notation #PCDATA stands for a “parsed character string”. In BNF notation, the grammar becomes

```
<PARTSLIST> ::= <ITEM>|<ITEM><PARTSLIST>
<ITEM> ::= <PARTNUMBER> <USEDINLIST>
<PARTNUMBER> ::= <identifier>
<USEDINLIST> ::=  $\lambda$  | <USEDIN>|<USEDIN><USEDINLIST>
<USEDIN> ::= <MAKE><MODEL>
<MAKE> ::= <identifier>
<MODEL> ::= <identifier>
<identifier> ::= [as defined in Example 55]
```

²“Panini-Backus Form Suggested,” Ingberman, P. Z., *Communications of the ACM*, vol. 10, No. 3, 1967.

A Web browser parses the XML document to determine whether it is a valid instance of the language defined by its associated DTD, and if so, displays it. ●

Classes of Grammars

Before we identify some types of grammars, let's look at one more example.

EXAMPLE 57

Let L be the empty string λ together with the set of all strings consisting of an odd number n of 0s, $n \geq 3$. The grammar $G = (V, V_T, S, P)$ generates L where $V = \{0, A, B, E, F, W, X, Y, Z, S\}$, $V_T = \{0\}$, and the productions are

$$\begin{array}{lll} S \rightarrow FA & 0X \rightarrow X0 & 0Z \rightarrow Z0 \\ S \rightarrow FBA & Y0 \rightarrow 0Y & WBZ \rightarrow EB \\ FB \rightarrow F0EB0 & FX \rightarrow F0W & F \rightarrow \lambda \\ EB \rightarrow 0 & YA \rightarrow Z0A & A \rightarrow \lambda \\ EB \rightarrow XBY & W0 \rightarrow 0W & \end{array}$$

The derivation $S \Rightarrow FA \xRightarrow{*} \lambda\lambda = \lambda$ produces λ . The derivation

$$\begin{aligned} S &\Rightarrow FBA \\ &\Rightarrow F0EB0A \\ &\Rightarrow F0XBY0A \\ &\xRightarrow{*} FX0B0YA \\ &\xRightarrow{*} F0W0B0Z0A \\ &\xRightarrow{*} F00WBZ00A \\ &\Rightarrow F00EB00A \\ &\Rightarrow F00000A \\ &\xRightarrow{*} 00000 \end{aligned}$$

produces five 0s. Notice how X and Y , and also W and Z , march back and forth across the strings of 0s, adding one more 0 on each side. This activity is highly reminiscent of a Turing machine read-write head sweeping back and forth across its tape and enlarging the printed portion. ●

REMINDER

The erasing convention says how to do erasing IF erasing is to be done. It does not require the production $S \rightarrow \lambda$ in every grammar.

The preceding grammar allows the erasing productions $F \rightarrow \lambda$ and $A \rightarrow \lambda$. To generate any language containing λ , we have to be able to erase somewhere. In the following grammar types, we will limit erasing, if it occurs at all, to a single production of the form $S \rightarrow \lambda$, where S is the start symbol; if this production occurs, we will not allow S to appear on the right-hand side of any other productions. This restriction allows us to crank out λ from S as a special case and then get on with other derivations, none of which allow any erasing. Let's call this the **erasing convention**. The following definition defines three special types of grammars by further restricting the productions allowed.

● **DEFINITIONS** **CONTEXT-SENSITIVE, CONTEXT-FREE, AND REGULAR GRAMMARS; CHOMSKY HIERARCHY**

A grammar G is **context-sensitive (type 1)** if it obeys the erasing convention and if, for every production $\alpha \rightarrow \beta$ (except $S \rightarrow \lambda$), the word β is at least as long as the word α . A grammar G is **context-free (type 2)** if it obeys the erasing convention and for every production $\alpha \rightarrow \beta$, α is a single nonterminal. A grammar G is **regular (type 3)** if it obeys the erasing convention and for every production $\alpha \rightarrow \beta$ (except $S \rightarrow \lambda$), α is a single nonterminal and β is of the form t or tW , where t is a terminal symbol and W is a nonterminal symbol. This hierarchy of grammars, from type 0 to type 3, is called the **Chomsky hierarchy**.

In a context-free grammar, a single nonterminal symbol on the left of a production can be replaced wherever it appears by the right side of the production. In a context-sensitive grammar, a given nonterminal symbol can perhaps be replaced only if it is part of a particular string (context)—hence the names *context-free* and *context-sensitive*. It is clear that any regular grammar is also context-free, and any context-free grammar is also context-sensitive. The grammar of Example 49 is regular (the two productions have the single nonterminal S on the left, and on the right either 1—a terminal—or $0S$ —a terminal followed by a nonterminal). Both grammars of Example 51 are context-free but not regular (again the single nonterminal S appears on the left of all productions, but the right sides consist of three symbols or two nonterminals or two terminals, respectively). The grammar of Example 52 is context-sensitive but not context-free (the productions do not shrink any strings, but some left sides have multiple symbols). The grammars of Example 53 and Example 54 are context-free but not regular (for example, the first three productions of Example 54 violate the requirement for a regular grammar). Finally, the grammar of Example 57 is a type 0 grammar, but it is not context-sensitive (for example, the production $EB \rightarrow 0$ is a “shrinking” production; also, the erasing convention is violated).

● **DEFINITION** **LANGUAGE TYPES**

A language is **type 0 (context-sensitive, context-free, or regular)** if it can be generated by a type 0 (context-sensitive, context-free, or regular) grammar.

Because of the relationships among the four grammar types, we can classify languages as shown in Figure 9.26. Thus, any regular language is also context-free because any regular grammar is also a context-free grammar, and so on. However, although it turns out to be true, we do not know from what we have done that these sets are properly contained in one another. For example, the language L described in Example 57 was generated in that example by a grammar that was type 0 but not context-sensitive, but that does not imply that L itself falls into that category. Different grammars can generate the same language.

Formal Language Hierarchy

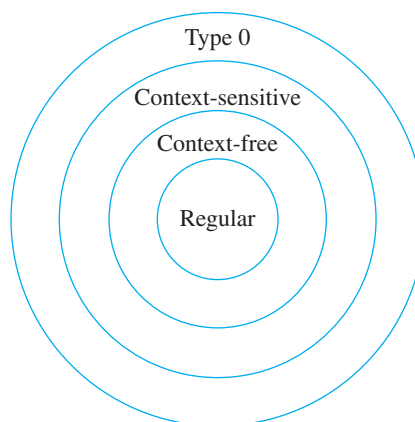


Figure 9.26

• **DEFINITION EQUIVALENT GRAMMARS**

Two grammars are **equivalent** if they generate the same language.

EXAMPLE 58

Example 57 gave a grammar G to generate a language L , which is λ together with all odd-length strings of 0s of length at least 3. We will now give three more grammars equivalent to G . (See if you agree that each of these grammars also generates L .)

$G_1 = (V, V_T, S, P)$ where $V = \{0, A, B, S\}$, $V_T = \{0\}$, and the productions are

$$\begin{array}{ll} S \rightarrow \lambda & AB \rightarrow 00 \\ S \rightarrow ABA & 0A \rightarrow 000A \\ & A \rightarrow 0 \end{array}$$

G_1 is context-sensitive but not context-free.

$G_2 = (V, V_T, S, P)$ where $V = \{0, A, S\}$, $V_T = \{0\}$, and the productions are

$$\begin{array}{ll} S \rightarrow \lambda & A \rightarrow 00A \\ S \rightarrow 00A & A \rightarrow 0 \end{array}$$

G_2 is context-free but not regular.

$G_3 = (V, V_T, S, P)$ where $V = \{0, A, B, C, S\}$, $V_T = \{0\}$, and the productions are

$$\begin{array}{ll} S \rightarrow \lambda & B \rightarrow 0 \\ S \rightarrow 0A & B \rightarrow 0C \\ A \rightarrow 0B & C \rightarrow 0B \end{array}$$

G_3 is regular.

Thus, when all is said and done, L is a regular language. •

PRACTICE 65 Give the derivation of 00000 in G_1 , G_2 , and G_3 .

This is a somewhat confusing point and worth repeating. Because a given language can be generated by multiple grammars, if you find a grammar G for a language L that is type x but not type y , that does not necessarily make L type x but not type y . There might be a different grammar that also generates L and is a type y grammar.

Formal Languages and Computational Devices

The language L of Example 57 can be described by the regular expression $\lambda \vee (000)(00)^*$, so L is a regular set. From Example 58, L is also a regular language. It is not coincidental that a regular set turned out to be a regular language. It can be shown that for any finite-state machine, the set it recognizes is a regular language. It can also be shown that for any regular language, there is a finite-state machine that recognizes exactly that language. (In the proofs of these results, the productions of a regular grammar correspond to the state transitions of a finite-state machine.) Hence those sets recognized by finite-state machines—the regular sets—correspond to regular languages. Therefore the class of sets recognized by a computational device of limited capacity coincides with the most restricted class of languages.

On the other end of the spectrum, the most general computational device is the Turing machine and the most general language is a type 0 language. As it happens, the sets recognized by Turing machines correspond to type 0 languages.

There are computational devices with capabilities midway between those of finite-state machines and those of Turing machines. These devices recognize exactly the context-free languages and the context-sensitive languages, respectively.

The type of device that recognizes the context-free languages is called a **pushdown automaton**, or **pda**. A pda consists of a finite-state unit that reads input from a tape and controls activity in a stack. Symbols from some alphabet can be pushed onto or popped off of the top of the stack. The finite-state unit in a pda, as a function of the input symbol read, the present state, and the top symbol on the stack, has a finite number of possible next moves. The moves are of the following types:

1. Go to a new state, pop the top symbol off the stack, and read the next input symbol.
2. Go to a new state, pop the top symbol off the stack, push a finite number of symbols onto the stack, and read the next input symbol.
3. Ignore the input symbol being read, manipulate the stack as above, but do not read the next input symbol.

A pda has a choice of next moves, and it recognizes the set of all inputs for which some sequence of moves exists that causes it to empty its stack. It can be shown that any set recognized by a pda is a context-free language, and conversely.

The type of device that recognizes the context-sensitive languages is called a **linear bounded automaton**, or **lba**. An lba is a Turing machine whose read-write head is restricted to a portion of the tape that is no longer than a constant multiple of the length of the original input; in addition, at each step it has a choice of possible next moves. An lba recognizes the set of all inputs for which some sequence

of moves exists that causes it to halt in a final state. Any set recognized by an lba can be shown to be a context-sensitive language, and conversely.

Figure 9.27 shows the relationship between the hierarchy of languages and the hierarchy of computational devices.

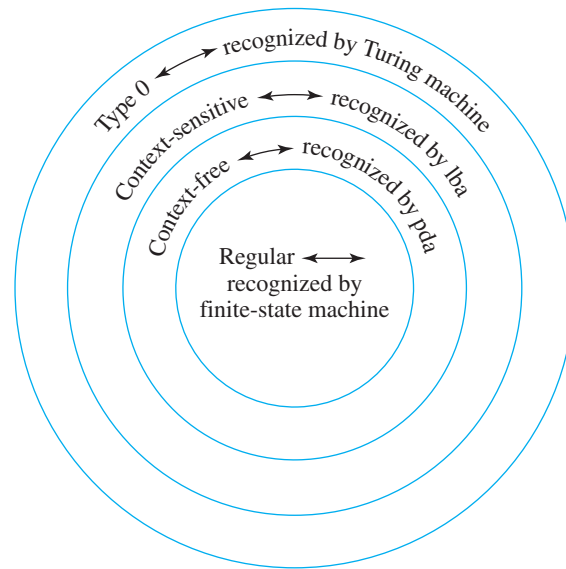


Figure 9.27

Context-Free Grammars

Context-free grammars are important for three reasons. Context-free grammars seem to be the easiest to work with, since they allow replacing only one symbol at a time. Furthermore, many programming languages are defined such that sections of syntax, if not the whole language, can be described by context-free grammars. Finally, a derivation in a context-free grammar has a nice graphical representation called a **parse tree**.

EXAMPLE 59

The grammar of Example 54 is context-free. The word $d2q$ can be derived as follows: **identifier** $\Rightarrow$ **identifier letter** $\Rightarrow$ **identifier digit letter** $\Rightarrow$ **letter digit letter** $\Rightarrow$ **d digit letter** $\Rightarrow$ **$d2$ letter** $\Rightarrow$ **$d2q$** . We can represent this derivation as a tree with the start symbol for the root. When a production is applied to a node, that node is replaced at the next lower level of the tree by the symbols in the right-hand side of the production used. A tree for the derivation appears in Figure 9.28.

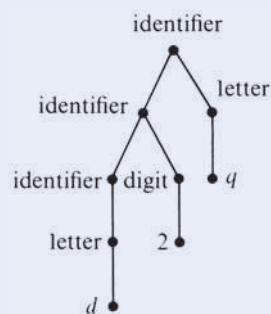


Figure 9.28

PRACTICE 66 Draw a parse tree for the word *m34s* in the grammar of Example 54.

Suppose that a context-free grammar G describes a programming language. The programmer uses the rules of G to generate legitimate strings of symbols, that is, words in the language. Here we may think of a word as corresponding to a program instruction. Thus, a word consists of various subwords, for example, identifiers, operators, and key words for the language. The program instructions are fed into the compiler for the language so that the program can be translated into machine language code for the computer. The compiler must decide whether each program instruction is legitimate in the language. In a sense, the compiler has to undo the process that the programmer used to construct the statement; that is, the compiler must start with the statement and decompose it to see whether it follows the rules of the language. This really entails two questions: Are the subwords themselves legitimate strings? Is the program instruction a legitimate way of grouping the subwords together?

Usually the set of legitimate subwords of a language can be described by a regular expression, and then a finite-state machine can be used to detect the subwords; the lexical analysis or scanner portion of the compiler handles this phase of compilation. (See Exercise 32 of Section 9.3 for the lexical analysis of ScrubOak.) If all goes well, the scanner then passes the program instruction, in the form of a string of legitimate subwords, to the syntax analyzer. The syntax analyzer determines whether the string is correct by trying to parse it (construct its parse tree).

Various parsing techniques, which we won't go into, have been devised. Given a string to be tested, one approach is to construct a tree by beginning with the start symbol, applying productions (keeping an eye on the "goal"—the given string) and ending with the goal string. This procedure is called **top-down parsing**. The alternative is to begin with the string, see what productions were used to create it, apply productions "backwards," and end with the start symbol. This process is called **bottom-up parsing**. The trick to either approach is to decide exactly which productions should be used.

EXAMPLE 60

Consider the context-free grammar G given by $G = (V, V_T, S, P)$ where $V = \{a, b, c, A, B, C, S\}$, $V_T = \{a, b, c\}$, and the productions are

$$\begin{array}{lll} S \rightarrow B & B \rightarrow C & A \rightarrow abc \\ S \rightarrow A & B \rightarrow ab & C \rightarrow c \end{array}$$

Suppose we want to test the string *abc*. A derivation for *abc* is $S \Rightarrow A \Rightarrow abc$. If we try a top-down parse, we might begin with

$$\begin{array}{c} S \\ \downarrow \\ B \end{array}$$

Then we have to detect that this will not work and try something else. If we try a bottom-up parse, we might begin with

$$\begin{array}{c} BC \\ \uparrow \\ abc \end{array}$$

Then we have to detect that this will not work and try something else. Parsing techniques automate this process and attempt to minimize the amount of false starts and backtracking required. ●

Notice the distinction between *generating* members of a set, which the programmer does, and *deciding* membership in a set, which the compiler does. Since we ask the compiler to decide membership in a set, a decision algorithm must exist for the set. It turns out that decision algorithms do exist for context-free languages, another point in their favor.

SECTION 9.5 REVIEW

TECHNIQUES

- Describe $L(G)$ for a given grammar G .
- Define a grammar to generate a given language L .
- Construct parse trees in a context-free grammar.

MAIN IDEAS

- A grammar G is a generating mechanism for its language $L(G)$.
- Formal languages were developed in an attempt to describe correct syntax for natural languages; although this attempt has largely failed because of the complexity of natural languages, it has been quite successful for high-level programming languages.
- Special classes of grammars are defined by restricting the allowable productions.
- The various types of formal languages correspond to the sets recognized by various automata; in particular, (1) regular languages are the sets recognized by finite-state machines, (2) context-free languages are the sets recognized by pushdown automata, (3) context-sensitive languages are the sets recognized by linear bounded automata, and (4) type 0 languages are the sets recognized by Turing machines.
- Derivations in context-free grammars can be illustrated by parse trees.
- A compiler for a context-free programming language checks correct syntax by parsing.

EXERCISES 9.5

1. Describe $L(G)$ for the grammar $G = (V, V_T, S, P)$ where $V = \{a, A, B, C, S\}$, $V_T = \{a\}$, and P consists of

$$\begin{array}{ll} S \rightarrow A & B \rightarrow A \\ A \rightarrow BC & aC \rightarrow \lambda \\ A \rightarrow a & \end{array}$$

What type of grammar is this?

2. Describe $L(G)$ for the grammar $G = (V, V_T, S, P)$ where $V = \{0, 1, A, B, S\}$, $V_T = \{0, 1\}$, and P consists of

$$\begin{array}{lll} S \rightarrow 0A & A \rightarrow 1BB & B \rightarrow 01 \\ S \rightarrow 1A & & B \rightarrow 11 \end{array}$$

What type of grammar is this?

3. Describe $L(G)$ for the grammar $G = (V, V_T, S, P)$ where $V = \{0, 1, A, B, S\}$, $V_T = \{0, 1\}$, and P consists of

$$\begin{array}{lll} S \rightarrow 0 & A \rightarrow 1B & B \rightarrow 0A \\ S \rightarrow 0A & & B \rightarrow 0 \end{array}$$

What type of grammar is this?

4. Describe $L(G)$ for the grammar $G = (V, V_T, S, P)$ where $V = \{0, 1, A, S\}$, $V_T = \{0, 1\}$, and P consists of

$$\begin{array}{ll} S \rightarrow 0S & A \rightarrow 1A \\ S \rightarrow 11A & A \rightarrow 1 \end{array}$$

What type of grammar is this?

5. Find a regular grammar that generates the language of Exercise 1.
 6. Find a regular grammar that generates the language of Exercise 2.
 7. Find a regular grammar that generates the language of Exercise 3.
 8. Find a regular grammar that generates the language of Exercise 4.
 9. Describe $L(G)$ for the grammar $G = (V, V_T, S, P)$ where $V = \{a, b, A, B, S\}$, $V_T = \{a, b\}$, and P consists of

$$\begin{array}{l} S \rightarrow aA \\ S \rightarrow \lambda \\ A \rightarrow bS \end{array}$$

10. Describe $L(G)$ for the grammar $G = (V, V_T, S, P)$ where $V = \{0, 1, A, S\}$, $V_T = \{0, 1\}$, and P consists of

$$\begin{array}{l} S \rightarrow 0 \\ S \rightarrow ASA \\ A \rightarrow 1 \end{array}$$

11. Describe $L(G)$ for the grammar $G = (V, V_T, S, P)$ where $V = \{a, b, A, B, S\}$, $V_T = \{a, b\}$, and P consists of

$$\begin{array}{ll} S \rightarrow AB & AB \rightarrow AAB \\ A \rightarrow a & AB \rightarrow ABB \\ B \rightarrow b & \end{array}$$

What type of grammar is G ? Find a regular grammar G' that generates $L(G)$.

12. Describe $L(G)$ for the grammar $G = (V, V_T, S, P)$ where $V = \{a, b, c, A, B, C, R, T, S\}$, $V_T = \{a, b, c\}$, and P consists of

$$\begin{array}{ll} S \rightarrow ARBT & T \rightarrow cT \\ R \rightarrow ARB & T \rightarrow c \\ RB \rightarrow ACBB & A \rightarrow a \\ CB \rightarrow BC & B \rightarrow b \\ CT \rightarrow cT & \end{array}$$

What type of grammar is G ? Find a context-free grammar G' that generates $L(G)$. Explain why it is not possible to find a regular grammar that generates $L(G)$.

13. Write the productions of the following grammars in BNF:

- a. G in Exercise 2
- b. G in Exercise 3
- c. G in Exercise 4

14. Write the productions of the following grammars in BNF:

- a. G in Exercise 10
- b. G_3 in Example 58

15. A grammar G is described in BNF as

$$\langle S \rangle ::= 1|1\langle S \rangle|\langle S \rangle 00$$

- a. Find a regular expression for $L(G)$.
- b. Find a regular grammar for $L(G)$.

16. A grammar G is described in BNF as

$$\langle S \rangle ::= 01|0\langle S \rangle|\langle S \rangle 1$$

- a. Find a regular expression for $L(G)$.
- b. Find a regular grammar for $L(G)$.

17. Find a grammar that generates the set of all strings of well-balanced parentheses.

18. English words are translated into “pig latin” by the following two rules:

- 1. If a word begins with a vowel, add the suffix “yay”.
- 2. If a word begins with a string of one or more consonants, move the string of leading consonants to the back of the word and then add the suffix “ay”.
- a. What is the pig latin translation of the word “apple”?
- b. What is the pig latin translation of the word “monkey”?
- c. What is the pig latin translation of the word “chain”?
- d. Find a grammar that generates all legal pig latin words.
- e. Using your grammar from part d, generate the pig latin word that was the answer to part c.

19. A word w in V^* is a palindrome if $w = w^R$, where w^R is the reverse of the string w . A language L is a palindrome language if L consists entirely of palindromes. Find a grammar that generates the set of all palindromes over the alphabet $\{a, b\}$.

20. a. Let L be a palindrome language (see Exercise 19). Prove that $L^R = \{w^R | w \in L\}$ is a palindrome language.

b. Let w be a palindrome. Prove that the language described by the regular expression w^* is a palindrome language.

21. Find a regular grammar that generates the language $L = 11(0 \vee 1)^*$.

22. Find a regular grammar that generates the language $L = (0 \vee 1)^*01$.

23. Find a grammar that generates the language $L = \{1^{2n} | n \geq 0\}$.

24. Find a regular expression for the language of Exercise 23. Is your grammar from Exercise 23 regular? If not, write a regular grammar that generates L .

25. Find a context-free grammar that generates the language $L = \{0^n 1^n | n \geq 0\}$.

26. Find a context-free grammar that generates the language $L = \{ww^R \mid w \in \{0, 1\}^*\}$ and w^R is the reverse of the string w .
27. Find a context-free grammar that generates the language L where L consists of the set of all nonempty strings of 0s and 1s with an equal number of 0s and 1s.
28. Find a context-free grammar that generates the language L where L consists of the set of all nonempty strings of 0s and 1s with twice as many 0s as 1s.
29. Find a grammar that generates the language $L = \{0^{2^i} \mid i \geq 0\}$.
30. Find a grammar that generates the language $L = \{0^n 1^{2^n} 0^n \mid n \geq 0\}$.
31. Find a grammar that generates the language $L = \{ww \mid w \in \{0, 1\}^*\}$.
32. Find a grammar that generates the language $L = \{a^{n^2} \mid n \geq 1\}$. (By Exercise 38, L is not a context-free language, so your grammar cannot be too simple.)
33. Draw parse trees for the following words:
 - a. 111111 in the grammar G of Example 51
 - b. 111111 in the grammar G' of Example 51
34. Draw parse trees for the following words:
 - a. 011101 in the grammar G of Exercise 2
 - b. 00111111 in the grammar G of Exercise 4
35. Consider the context-free grammar $G = (V, V_T, S, P)$ where $V = \{0, 1, A, S\}$, $V_T = \{0, 1\}$, and P consists of

$$\begin{aligned} S &\rightarrow A1A \\ A &\rightarrow 0 \\ A &\rightarrow A1A \end{aligned}$$

Draw two distinct parse trees for the word 01010 in G . A grammar in which a word has more than one parse tree is *ambiguous*.

36. Ambiguity in a grammar (see Exercise 35) that describes a programming language is an undesirable trait because the compiler is unable to uniquely parse the programming instruction. Ambiguity in a natural language can be similarly confusing. The following two instructions were posted beside an escalator.

“Shoes must be worn.”

“Dogs must be carried.”

Give two possible interpretations for each instruction.

37. Show that for any context-free grammar G there exists a context-free grammar G' in which for every production $\alpha \rightarrow \beta$, β is a longer string than α , $L(G') \subseteq L(G)$ and $L(G) - L(G')$ is a finite set.
38. Following is the *pumping lemma* for context-free languages. Let L be any context-free language. Then there exists some constant k such that for any word w in L with $|w| \geq k$, w can be written as the string $w_1 w_2 w_3 w_4 w_5$ with $|w_2 w_3 w_4| \leq k$ and $|w_2 w_4| \geq 1$. Furthermore, the word $w_1 w_2^i w_3 w_4^i w_5 \in L$ for each $i \geq 0$.
 - a. Use the pumping lemma to show that $L = \{a^n b^n c^n \mid n \geq 1\}$ is not context-free.
 - b. Use the pumping lemma to show that $L = \{a^{n^2} \mid n \geq 1\}$ is not context-free.